Sorting as Gradient Flow on the Permutohedron

Jonathan Robert Landers
jonathan.robert.landers@gmail.com

Abstract

We investigate how sorting algorithms navigate the complexity of permutation space. Our main contribution is a continuous-time model that casts the task as directed motion. A quadratic potential on the permutohedron generates an ambient gradient flow that contracts toward the fixed sorted vertex $v_s = (1, 2, \ldots, n)$. This dynamical picture is set against two discrete descriptions of the same problem. One follows adjacent-swap paths along the 1-skeleton, while the other uses comparison half-spaces to refine the feasible order types. Together, they provide the combinatorial foils used to evaluate the continuous trajectory. Comparisons remove informational ambiguity, whereas the flow removes metric distance. The two mechanisms are complementary. To support this analysis, we present decision-tree arguments and local paths with $\Theta(n^2)$ behavior. We also show that the quadratic potential decreases strictly under every inversion-removing adjacent swap and formulate global half-space constraints on candidate rank maps. The maximal fixed-threshold relaxation time is determined exactly by the Euclidean diameter of the permutohedron, and the product of this relaxation time with the dimension is $\Theta(n \log n)$. Under a normalized comparison clock, this intrinsic geometric quantity has the same asymptotic scale as classical optimal comparison sorting.

MSC 2020: 68Q25, 52B11, 90C52
Keywords: sorting algorithms, permutohedron, gradient flow, computational complexity

GitHub repository Project website

1 Introduction

Consider the fundamental problem of sorting a list

$$L = [\ell_1, \ell_2, \ldots, \ell_n]. \tag{1}$$

The elements are taken to lie in a totally ordered set so that their relative order is well-defined. Rather than viewing sorting merely as a sequence of symbolic operations, it is useful to regard it as navigation through a large structured space. Each input determines one of $n!$ possible order types, and comparisons locate the correct one. Computational models then reveal distinct notions of motion, distance, and progress within that space.

Notation. The key issue is separating an item's original identity from its current position. We therefore keep four related objects separate. The integer i is the original index of an item, and ℓ_i is the value at that index. Since the entries are assumed pairwise distinct, define the original rank map

$$\rho(i) = \operatorname{rank}(\ell_i), \tag{2}$$

where rank 1 is smallest and rank n is largest. At algorithmic time t, let $p_t(k)$ be the original index of the item currently occupying array position k. The rank word plotted on the permutohedron is

$$x_t(k) = \rho(p_t(k)). \tag{3}$$

Thus x_t records the ranks currently sitting in array positions, and initially

$$x_0 = (\rho(1), \rho(2), \ldots, \rho(n)). \tag{4}$$

The rank word is used here as an ex post representation of the input order type. Sorting moves the rank word x_0 toward $v_s = (1, 2, \ldots, n)$. The sorted index order is a different object. Here $\sigma(k)$ is the original index of the item occupying sorted position k, so $\sigma = \rho^{-1}$, and

$$[\ell_{\sigma(1)}, \ell_{\sigma(2)}, \ldots, \ell_{\sigma(n)}] \tag{5}$$

is sorted. For example, if $L = [3, 1, 2]$, then $\rho = (3, 1, 2)$ but $\sigma = (2, 3, 1)$. These differ in general. Table 1 later traces all of the notation through the running example $L = [5, 4, 2]$.

We write τ for a permutation of the index set $\{1, 2, \ldots, n\}$, acting on L by

$$\tau(L) = [\ \ell_{\tau(1)},\ \ell_{\tau(2)},\ \ldots,\ \ell_{\tau(n)}\ ]. \tag{6}$$

Setting. A brute-force baseline is to enumerate all $n!$ permutations $\tau \in S_n$ and select σ, the permutation that produces the sorted list. This costs $\Theta(n \cdot n!)$, since each candidate ordering requires $O(n)$ time to check. We use standard asymptotic notation as in [1, 2]. Bounds are worst-case unless stated otherwise, and randomized costs are denoted by $\mathbb{E}[T(n)]$. Decision-tree logarithms are taken in base 2, while natural logarithms are used in exact continuous-time formulas. The choice of base does not affect asymptotic orders.

Within this framework, established sorting algorithms such as MergeSort, HeapSort, and QuickSort achieve a remarkable improvement, reducing the search from factorial to log-linear scaling. By contrast, local-swap methods such as BubbleSort use only adjacent comparisons and swaps, yielding quadratic $\Theta(n^2)$ behavior. MergeSort and HeapSort guarantee $O(n \log n)$ worst-case performance, while QuickSort attains the same bound in expectation. Each exploits regularities in the input, effectively pruning large regions of permutation space.

This striking efficiency has long inspired curiosity about the connection between organization and computational tractability. Foundational work by Knuth [3], Shannon [4], and others, along with recent combinatorial and geometric perspectives [5–7], reflects ongoing efforts to understand how structure constrains and organizes computation.

Main contribution. We place three views of sorting in a common permutohedral state space. Adjacent swaps traverse edges, comparison half-spaces eliminate candidate rank maps, and a quadratic potential generates an exactly solvable ambient gradient flow toward the sorted vertex. The potential decreases strictly under every inversion-removing adjacent swap. For the continuous model, Theorem 4.2 gives the straight-line contraction, Proposition 4.3 determines the exact maximal fixed-threshold relaxation time from the diameter of $\mathcal{P}_n$, and Theorem 4.4 proves the intrinsic factorization $\dim(\mathcal{P}_n) T_\varepsilon^{\max}(n) = \Theta(n \log n)$ for fixed $\varepsilon > 0$. Corollary 4.5 compares this independently proved geometric quantity with the normalized classical comparison scale. The decision-tree argument remains the rigorous computational source of the comparison lower bound; the flow supplies a geometric correspondence at the same asymptotic scale.

We begin by establishing the decision-tree complexity scale, then reinterpret comparisons as linear constraints acting on the permutohedron, and finally introduce a gradient-flow model that renders geometric contraction concrete.

2 Information-Theoretic Decision-Tree Lower Bound

The information-theoretic limit for comparison sorting follows by modeling the process as a binary decision tree, whose leaves distinguish the possible strict order types, equivalently the required output permutations.

Lemma 2.1 (Decision-Tree Lower Bound)**.** *Let T be a binary decision tree that correctly sorts n elements using comparisons. Then the height h of T satisfies*

$$h \geq \log_2(n!). \tag{7}$$

Proof. A deterministic comparison sort must distinguish all $n!$ strict input orderings, and distinct order types cannot share a leaf, which prescribes a single output permutation. Thus a tree of height h must satisfy

$$2^h \geq n!. \tag{8}$$

Taking logarithms and using Stirling's approximation gives

$$h \geq \log_2(n!) = n \log_2 n - O(n) = \Omega(n \log n). \tag{9}$$

Classical comparison sorts attain the matching upper bound. □

This information-theoretic picture also has an enumerative counterpart. Harris, Kretschmann, and Martínez Mori [6] have shown that the total number of comparisons made by QuickSort over all $n!$ input orderings coincides with the number of parking-preference lists admitting exactly $n - 1$ lucky cars. This identity further underscores how comparison structure governs the complexity of sorting.

Figure 1 illustrates the decision tree for sorting three elements, instantiated with the input $L = (\ell_1 = 5, \ell_2 = 4, \ell_3 = 2)$. The leaves are labeled by candidate sorting permutations of the original indices, with the reordered list shown for each outcome. A highlighted path records the three comparisons performed on this input and terminates at $\sigma = (3, 2, 1)$, which yields the sorted output $[2, 4, 5]$. Thus the example realizes $\lceil \log_2(3!) \rceil = 3$, in agreement with the lower bound. Knuth's seminal analysis established this information-theoretic limit for sorting algorithms [3].

The decision-tree model fixes the comparison complexity, but it does not explain the geometry of the space being carved. This distinction matters. Algorithms restricted to local adjacent comparisons, such as BubbleSort, resolve one inversion at a time and have worst-case cost $\Theta(n^2)$. Arbitrary comparisons can approach the balanced $\Theta(n \log n)$ bound. In the next section we express this contrast on the permutohedron by interpreting comparison outcomes as linear inequalities. Sorting then appears not only as logical branching but as structured contraction of the candidate set until a single rank map remains.

3 Geometric Perspective on the Permutohedron and Half-Space Constraints

The same sorting process now admits two geometric readings on the permutohedron. Adjacent swaps trace paths along its edges and produce the familiar $\Theta(n^2)$ behavior of local sorting algorithms. Comparisons act differently. As linear inequalities, they remove incompatible order types from consideration. Thus one geometry describes local motion, while the other captures information refinement on the way to the optimal $\Theta(n \log n)$ comparison bound.

Definition 3.1 (Permutohedron)**.** Let

$$v_s = (1, 2, \ldots, n) \in \mathbb{R}^n. \tag{10}$$

The permutohedron $\mathcal{P}_n$ is defined as

$$\mathcal{P}_n = \mathrm{conv}\{\pi(v_s) \mid \pi \in S_n\}. \tag{11}$$

Its vertex set is $\mathcal{V}_n = \{\pi(v_s) \mid \pi \in S_n\}$. In this paper these vertices are read as rank words. The coordinate in position k is the rank currently occupying that position.

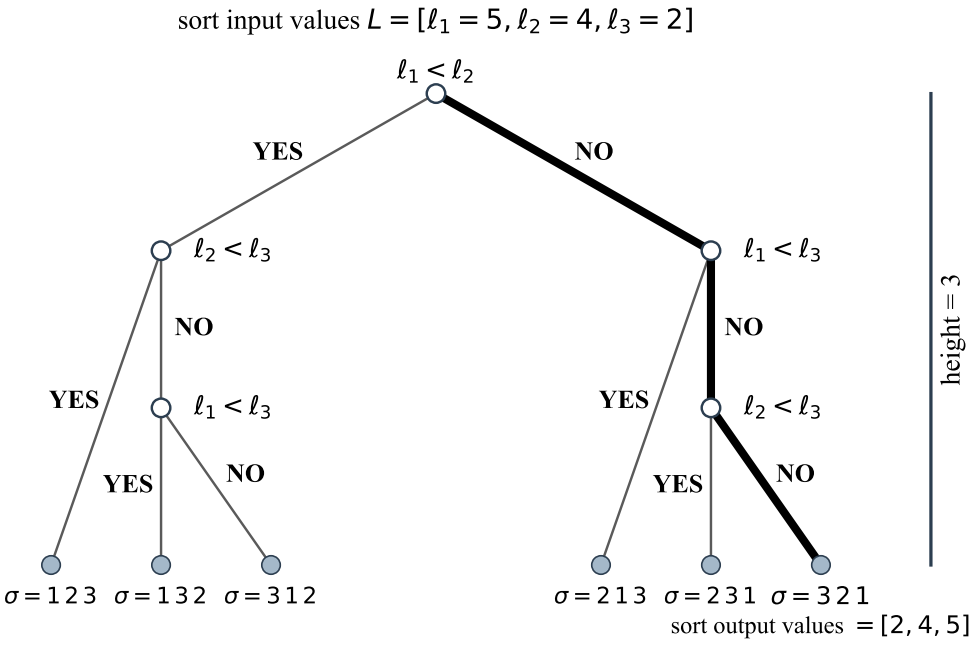

Figure 1: Decision tree for sorting $L = (\ell_1 = 5, \ell_2 = 4, \ell_3 = 2)$. Internal nodes are comparisons and leaves are candidate sorting permutations. The highlighted path identifies $\sigma = (3, 2, 1)$, yielding $[2, 4, 5]$ in three comparisons, matching $\lceil \log_2(3!) \rceil = 3$. Later permutohedron figures use rank-word coordinates rather than these index-order leaf labels.

Proposition 3.2 (Dimensionality and Affine Containment)**.** *The permutohedron $\mathcal{P}_n$ is an $(n-1)$-dimensional polytope contained in the affine hyperplane*

$$H = \left\{ x \in \mathbb{R}^n \,\middle|\, \sum_{i=1}^{n} x_i = \frac{n(n+1)}{2} \right\}. \tag{12}$$

Proof. For any permutation $\pi \in S_n$,

$$\sum_{i=1}^{n} (\pi(v_s))_i = \sum_{i=1}^{n} i = \frac{n(n+1)}{2}. \tag{13}$$

Thus, every vertex of $\mathcal{P}_n$ lies in the hyperplane H.

To see that these vertices span H, note that for each adjacent transposition $(i, i+1)$,

$$(i, i+1)(v_s) - v_s = \pm(e_{i+1} - e_i), \tag{14}$$

with the sign depending only on the convention for writing the permutation action. The vectors $e_{i+1} - e_i$ for $i = 1, \ldots, n-1$ form a basis of the subspace

$$\left\{ x \in \mathbb{R}^n : \sum_{i=1}^{n} x_i = 0 \right\}. \tag{15}$$

Since adjacent transpositions generate all permutations, it follows that the set $\{\pi(v_s) : \pi \in S_n\}$ affinely spans the entire hyperplane H.

Because H has dimension $n-1$, the permutohedron $\mathcal{P}_n$ is an $(n-1)$-dimensional polytope. □

3.1 Adjacent-Swap Walks and Their Geometric Interpretation

Before introducing the linear-constraint formulation, it is helpful to visualize how rank-word vertices relate to one another on the permutohedron $\mathcal{P}_n$. Because adjacent transpositions generate S_n, they appear naturally as edges of the 1-skeleton (Cayley graph) of $\mathcal{P}_n$. Traversing these edges resolves one inversion at a time. It gives a literal geometric realization of sorting as motion on a polytope and mirrors the stepwise progression in the decision-tree example for the input $L = (\ell_1, \ell_2, \ell_3) = (5, 4, 2)$. Shortest paths from the identity to the reverse permutation in this graph are the sorting networks studied by Angel et al. [8]. The walk diagrams use an inverse labeling of the standard permutohedron so that exchanging neighboring array positions corresponds to traversing an edge. This differs from direct coordinate labeling by permutation inversion.

Figure 2 depicts this adjacent-swap path for $n = 3$ in two complementary forms. One is a flattened 2-dimensional schematic and the other is a full 3-dimensional embedding of $\mathcal{P}_3$. In both views, the highlighted trajectory connects the reverse rank word $(3, 2, 1)$ to the sorted rank word $(1, 2, 3)$ by successive neighbor exchanges. These local moves provide a geometric model for the $\Theta(n^2)$ behavior of adjacent-swap sorting algorithms. The low-dimensional example is used only because $\mathcal{P}_3$ can be drawn directly. The definitions, constraints, and flow estimates below are stated for arbitrary n.

An edge path records how the current arrangement changes. A comparison history instead determines which original rank maps remain possible. When its outcomes are recorded as inequalities between candidate ranks of the original items, they define half-spaces whose intersection with $\mathcal{V}_n$ narrows the search space. Proposition 3.3 connects this refinement to the information-theoretic lower bound of $\Omega(n \log n)$ comparisons.

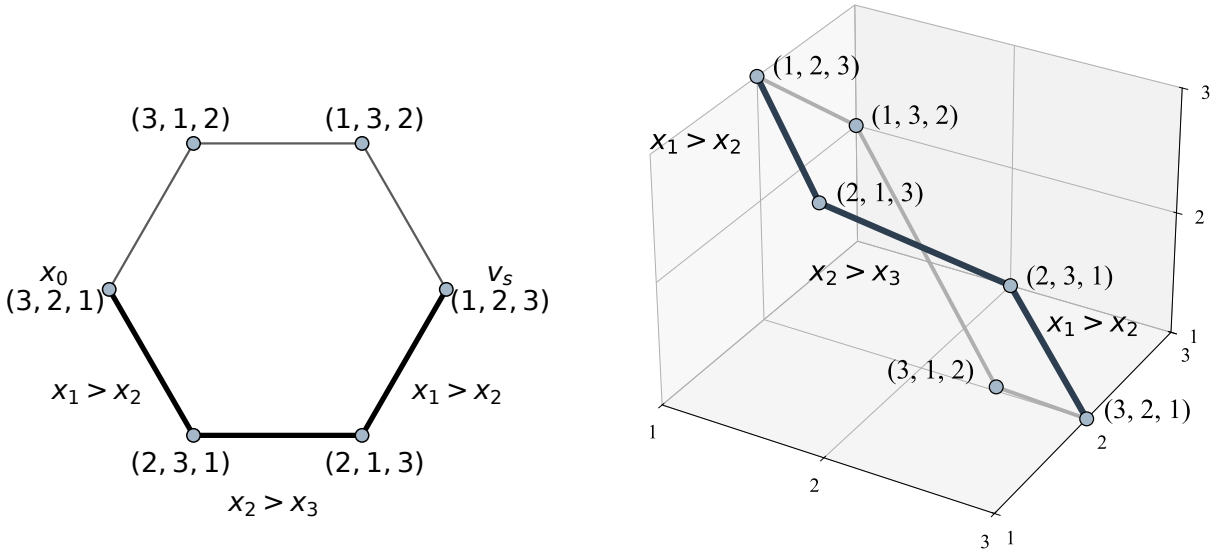

Figure 2: Adjacent-position-swap motion on an inverse-labeled permutohedron $\mathcal{P}_3$. **Left panel.** A 2D schematic of local traversal along the 1-skeleton. **Right panel.** The same array-rank path $(3, 2, 1) \to (2, 3, 1) \to (2, 1, 3) \to (1, 2, 3)$ in a 3D embedding. Each step exchanges neighboring positions, illustrating the local $\Theta(n^2)$ geometry of adjacent-swap sorting.

3.2 Linear Constraints and Information Refinement

Comparisons at different stages must be placed in a common coordinate frame. A comparison is made between two current positions, but the information it reveals concerns the two original items occupying those

positions. To combine all comparison outcomes into one fixed feasible set, the inequalities must therefore be recorded in original-item coordinates.

Let

$$r = (r_1, \ldots, r_n) \in \mathcal{V}_n \tag{16}$$

be a candidate original-index rank map, where r_i is the candidate rank of the original item ℓ_i. If at time t a comparison between positions a and b has the "less than" outcome, it records

$$r_{p_t(a)} < r_{p_t(b)}. \tag{17}$$

In plain language, if position a currently contains item i and position b contains item j, the comparison records $r_i < r_j$.

For a fixed comparison history C, let $\lhd_j$ be $<$ or $>$ according to the outcome of comparison j, made at time t_j between positions a_j and b_j. Its feasible candidate rank maps are

$$\mathcal{R}_n(C) = \left\{ r \in \mathcal{V}_n : r_{p_{t_j}(a_j)} \lhd_j r_{p_{t_j}(b_j)} \text{ for every comparison } j \right\}. \tag{18}$$

t	p_t	Current list	$x_t = \rho \circ p_t$	New constraint	$\lvert\mathcal{R}_3(C_t)\rvert$
0	$(1,2,3)$	$[5,4,2]$	$(3,2,1)$	none	6
1	$(2,1,3)$	$[4,5,2]$	$(2,3,1)$	$r_1 > r_2$	3
2	$(2,3,1)$	$[4,2,5]$	$(2,1,3)$	$r_1 > r_3$	2
3	$(3,2,1)$	$[2,4,5]$	$(1,2,3)$	$r_2 > r_3$	1

Table 1: Index bookkeeping for adjacent-swap sorting of $L = [5,4,2]$. Here $\rho = \sigma = (3,2,1)$ only because the reverse permutation is self-inverse. In general $\sigma = \rho^{-1}$ need not equal ρ.

Table 1 places the moving and fixed-coordinate descriptions side by side. Row t records the state after the first t comparisons and any resulting swaps, and C_t denotes the outcomes accumulated by that time. The left columns track motion. Here p_t records which original items occupy the array positions, and $x_t = \rho \circ p_t$ is the derived rank-word coordinate. The right columns track information in fixed original-item coordinates. Thus the rank word moves $321 \to 231 \to 213 \to 123$ while the feasible candidate set shrinks $6 \to 3 \to 2 \to 1$. In the final row, $\mathcal{R}_3(C_3) = \{\rho\}$ and $x_3 = v_s$. The general statement is as follows.

Proposition 3.3 (Comparison Constraints and Candidate Rank Maps)**.** *For an input with original rank map ρ, a complete comparison history C at a decision-tree leaf identifies that map.*

$$\mathcal{R}_n(C) = \{\rho\}. \tag{19}$$

Consequently, a correct comparison decision tree has worst-case depth at least $\log_2(n!) = \Omega(n \log n)$, while optimal comparison sorts supply the matching $O(n \log n)$ upper bound. If $\sigma = \rho^{-1}$ is the output permutation, then applying it converts the identified rank map to the sorted rank word.

$$\big(\rho(\sigma(1)), \ldots, \rho(\sigma(n))\big) = (1, \ldots, n) = v_s. \tag{20}$$

Proof. Along a fixed decision-tree path, the positions compared and the original items occupying them are determined by the preceding outcomes and operations. A candidate rank map r follows this path exactly when it satisfies every recorded inequality, so $\mathcal{R}_n(C)$ is precisely the set of order types compatible with the history.

At a leaf, the algorithm has fixed one output permutation. Correctness requires that this permutation sort every candidate reaching that leaf, but an original-index rank map r is sorted by the unique permutation r^{-1}. Thus only one candidate rank map can reach a correct leaf. Since the true map ρ follows the recorded path, $\mathcal{R}_n(C) = \{\rho\}$, and applying $\sigma = \rho^{-1}$ gives the displayed sorted rank word.

In the decision-tree model, each comparison has only two possible outcomes. Thus, at best, it can distinguish between two classes of remaining candidate rank words. For the purpose of proving a lower bound, we may therefore use the optimally balanced decision-tree ideal. In this ideal, each comparison splits the remaining feasible candidates as evenly as possible. This is the most optimistic binary refinement available to any comparison-based algorithm. Such a cut need not be exactly balanced, and arbitrary comparison hyperplanes may fail to halve either the vertices or the volume of the permutohedron.

In this ideal halving model, after k comparisons,

$$\frac{n!}{2^k} \le 1 \quad \Rightarrow \quad k \ge \log_2(n!) = \Omega(n \log n). \tag{21}$$

Classical algorithms achieve $O(n \log n)$ comparisons. Hence their constraints cut down the candidate original-index rank maps to exactly one feasible vertex, supplying the matching upper bound. □

Table 1 records a literal execution. Figure 3 instead gives a canonical fixed-coordinate illustration of the decision-tree halving heuristic. The inequalities $x_1 < x_2$ and $x_2 < x_3$ isolate the increasing chamber and hence the vertex $(1, 2, 3)$. For a concrete execution, comparison outcomes are first recorded in original-item coordinates and are then converted to sorted-output coordinates. Under this approximation, the panels represent exponential reduction in the number of feasible permutations. An actual comparison may not split the remaining vertices or polytope volume evenly.

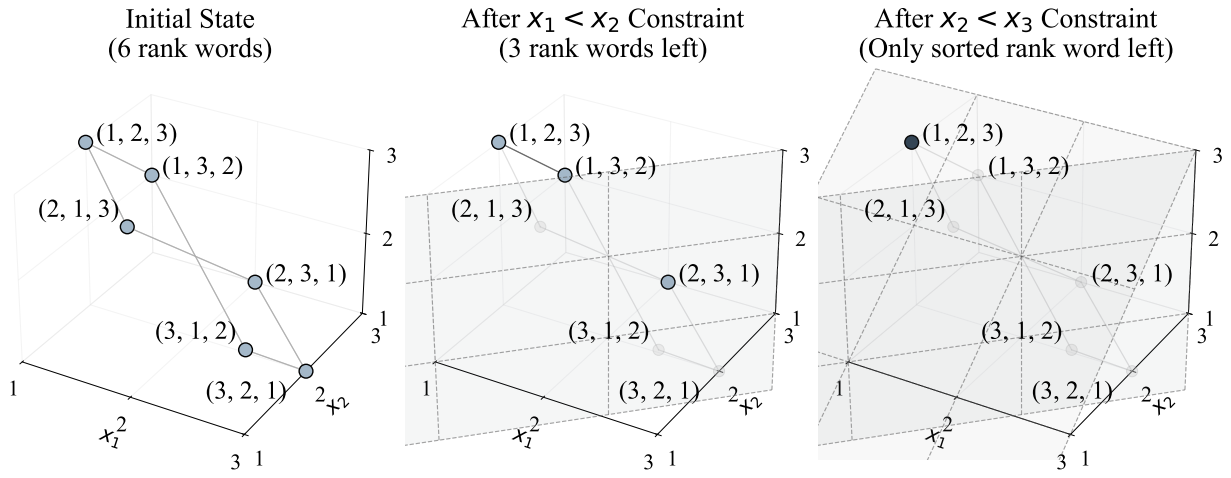

Figure 3: Canonical comparison half-spaces on $\mathcal{P}_3$. The cuts $x_1 < x_2$ and $x_2 < x_3$ isolate the increasing rank word. The picture illustrates chamber refinement rather than the literal adaptive comparison path of a particular input.

Several nearby approaches also connect sorting with geometry. Blelloch and Dobson relate comparison sorting to offline binary search trees through planar permutation representations and the log-interleave bound [5]. Lee et al. study stack-sorting simplices, subpolytopes encoding the geometry of a constrained sorting process [9]. Continuous relaxations include sorting-network formulations of permutation problems [10], entropy-regularized optimal-transport sorting [11], and differentiable sorting by projection onto

the permutahedron [12]. These approaches provide geometric models of particular sorting structures and optimization schemes.

The three viewpoints now on the table use distinct states and measures of progress. With adjacent exchanges, this is graph distance on the 1-skeleton of $\mathcal{P}_n$, equivalently the number of inversions to be removed. With arbitrary comparisons, the state is the set of feasible original-index rank maps, and disorder is the information still needed to isolate one of them. Euclidean distance in the standard embedding is a third object.

The central comparison therefore separates two geometrically different processes. First, comparisons eliminate incompatible order types until a single permutation remains. Second, once the target permutation is represented as a vertex, Euclidean gradient flow gives a continuous model of contraction toward it. These mechanisms are not identical. One is informational and the other metric. After comparisons are normalized by n, both exhibit logarithmic behavior. We now turn to the metric side of this picture.

4 Continuous Gradient Flow and the Classical $n \log n$ Scale

Starting from a fixed target, Theorem 4.2 gives the ambient gradient flow in closed form and quantifies its contraction time.

The closest dynamical antecedent is Brockett's double-bracket flow, which sorts a list through gradient-driven matrix diagonalization on an isospectral orbit whose moment image is the permutohedron [13, 14]. By contrast, the model here works directly in rank-word coordinates on the permutohedron and places fixed-target contraction alongside adjacent-swap motion and comparison half-spaces.

4.1 Setup and Definitions

Let

$$v_s = (1, 2, \ldots, n) \in \mathbb{R}^n \tag{22}$$

denote the vertex of the permutohedron corresponding to the sorted rank word. For the continuous relaxation, work in the standard Euclidean embedding of the permutohedron and define the potential

$$V(x) = \frac{1}{2}\|x - v_s\|_2^2 = \frac{1}{2}\sum_{i=1}^{n}(x_i - i)^2. \tag{23}$$

This is a Spearman-type squared rank displacement. It is not the only sorting metric. Adjacent swaps use inversion distance, whereas comparisons use the information remaining in the set of candidate rank maps. The role of V is to provide a smooth benchmark on $\mathcal{P}_n$. Note that $V(v_s) = 0$ and $V(x) > 0$ for $x \neq v_s$. Given an input list L with original rank map ρ, we take as initial state its rank word

$$x(0) = x_0 = (\rho(1), \rho(2), \ldots, \rho(n)) \in \mathcal{P}_n, \tag{24}$$

so that the relaxation carries this fixed geometric representative toward the sorted vertex v_s.

Proposition 4.1 (Adjacent-swap descent)**.** *Let $x \in \mathcal{V}_n$, and suppose $x_i = a > b = x_{i+1}$. If x' is obtained by swapping these adjacent inverted ranks, then*

$$V(x) - V(x') = a - b > 0. \tag{25}$$

Consequently, V is a strict Lyapunov function for adjacent inversion-removing sorting. If $\mathrm{Inv}(x)$ denotes the inversion number of x, then

$$\mathrm{Inv}(x) \leq V(x) \leq (n-1)\,\mathrm{Inv}(x). \tag{26}$$

Proof. Only coordinates i and $i+1$ change, so

$$V(x) - V(x') = \frac{1}{2}\left[(a-i)^2 + (b-i-1)^2 - (b-i)^2 - (a-i-1)^2\right] \tag{27}$$

$$= a - b. \tag{28}$$

Every adjacent inversion-removing swap decreases the inversion number by exactly one. Sorting x therefore takes $\mathrm{Inv}(x)$ such swaps, and each decrease in V lies between 1 and $n-1$. Summing the decreases along the path to v_s, where $V(v_s) = 0$, gives the stated bounds. □

Thus the same potential measures descent along the discrete adjacent-swap walk and the continuous flow introduced next. The two-sided bound parallels the Diaconis–Graham inequality between the Spearman footrule and the inversion count [15].

4.2 Exact Gradient-Flow Contraction

Theorem 4.2 (Ambient Euclidean Contraction)**.** *Let $x(0) \in \mathbb{R}^n$ be the initial rank word corresponding to an unsorted input list and let $v_s = (1, 2, \ldots, n)$ be the sorted vertex of the permutohedron $\mathcal{P}_n$. Define*

$$D_0 = \|x(0) - v_s\|_2^2. \tag{29}$$

Under the gradient flow

$$\dot{x}(t) = -\nabla V(x(t)) = v_s - x(t), \tag{30}$$

the following statements hold.

1. *The solution satisfies*

$$x(t) = v_s + \big(x(0) - v_s\big)e^{-t}, \tag{31}$$

$$\|x(t) - v_s\|_2 = \|x(0) - v_s\|_2 e^{-t}, \tag{32}$$

and hence

$$\|x(t) - v_s\|_2^2 = \|x(0) - v_s\|_2^2 e^{-2t}. \tag{33}$$

2. *For a threshold $0 < \varepsilon < \sqrt{D_0}$, the condition $\|x(t) - v_s\|_2 \le \varepsilon$ holds once*

$$t \ge \frac{1}{2}\ln\!\left(\frac{D_0}{\varepsilon^2}\right). \tag{34}$$

3. *For the reverse rank word $x(0) = (n, n-1, \ldots, 1)$,*

$$D_0 = \sum_{i=1}^{n} (n + 1 - 2i)^2 = \frac{n(n^2-1)}{3} = \Theta(n^3), \tag{35}$$

and therefore

$$V(x(0)) = \frac{D_0}{2} = \frac{n(n^2-1)}{6}. \tag{36}$$

For fixed $\varepsilon > 0$, the threshold time is $t = \Theta(\ln n)$.

Proof. We begin by deriving the gradient flow dynamics directly from the potential V. A straightforward computation shows that

$$\nabla V(x) = \big(x_1 - 1,\ x_2 - 2,\ \ldots,\ x_n - n\big). \tag{37}$$

Hence, the gradient flow is given by

$$\dot{x}(t) = -\nabla V(x(t)) = \big(1 - x_1(t),\ 2 - x_2(t),\ \ldots,\ n - x_n(t)\big). \tag{38}$$

Because the system decouples, each coordinate x_i satisfies the first-order linear differential equation

$$\dot{x}_i(t) = i - x_i(t), \tag{39}$$

with initial condition $x_i(0)$. Its solution is

$$x_i(t) = i + \big(x_i(0) - i\big)e^{-t}. \tag{40}$$

Collecting these coordinate solutions, we obtain

$$x(t) = v_s + \big(x(0) - v_s\big)e^{-t}, \tag{41}$$

so that

$$\|x(t) - v_s\|_2^2 = \|x(0) - v_s\|_2^2 e^{-2t}. \tag{42}$$

This establishes statement 1.

For statement 2, define the *initial disorder* $D_0 = \|x(0) - v_s\|_2^2$. The system is effectively sorted when

$$\|x(t) - v_s\|_2^2 = D_0 e^{-2t} \le \varepsilon^2. \tag{43}$$

Taking logarithms yields

$$t \ge \frac{1}{2} \ln\Big(\frac{D_0}{\varepsilon^2}\Big). \tag{44}$$

Reverse-permutation radius. If $x(0) = (n, n-1, \ldots, 1)$ then $x_i(0) - i = n + 1 - 2i$. The standard centered-square identity gives

$$D_0 = \sum_{i=1}^{n} \big(n + 1 - 2i\big)^2 = \frac{n(n^2 - 1)}{3} = \Theta(n^3). \tag{45}$$

Consequently the initial potential at the reverse rank word is

$$V(x(0)) = \frac{1}{2} D_0 = \frac{n(n^2 - 1)}{6}. \tag{46}$$

Hence $t = \Theta(\ln n)$. □

4.3 Maximal Relaxation Time

For $x_0 \in \mathcal{P}_n$, define the time to a Euclidean threshold

$$T_\varepsilon(x_0) = \inf\big\{t \ge 0 : \|x(t) - v_s\|_2 \le \varepsilon\big\}, \tag{47}$$

and define the worst-case relaxation time by

$$T_\varepsilon^{\max}(n) = \sup_{x_0 \in \mathcal{P}_n} T_\varepsilon(x_0). \tag{48}$$

Theorem 4.2 gives, including the case in which the initial point is already within the threshold,

$$T_\varepsilon(x_0) = \begin{cases} 0, & \|x_0 - v_s\|_2 \le \varepsilon, \\ \ln \dfrac{\|x_0 - v_s\|_2}{\varepsilon}, & \|x_0 - v_s\|_2 > \varepsilon. \end{cases} \tag{49}$$

Proposition 4.3 (Exact Maximal Relaxation Time)**.** *Let* $0 < \varepsilon < \operatorname{diam}(\mathcal{P}_n)$*. Under the flow of Theorem 4.2,*

$$T_\varepsilon^{\max}(n) = \ln \frac{\operatorname{diam}(\mathcal{P}_n)}{\varepsilon} = \frac{1}{2} \ln \left(\frac{n(n^2-1)}{3\varepsilon^2} \right). \tag{50}$$

In particular, for every fixed $\varepsilon > 0$*, as* $n \to \infty$*,*

$$T_\varepsilon^{\max}(n) = \frac{3}{2} \ln n + O(1). \tag{51}$$

Proof. The unsquared-distance identity in Theorem 4.2 gives

$$\|x(t) - v_s\|_2 = \|x_0 - v_s\|_2 e^{-t}. \tag{52}$$

Thus, whenever $\|x_0 - v_s\|_2 > \varepsilon$,

$$T_\varepsilon(x_0) = \ln \frac{\|x_0 - v_s\|_2}{\varepsilon}. \tag{53}$$

It remains to maximize the initial distance. The function $x \mapsto \|x - v_s\|_2^2$ is convex, so its maximum over the compact polytope $\mathcal{P}_n$ is attained at a vertex. Indeed, writing a point as a convex combination of vertices bounds its value by the largest value at those vertices. For a vertex $\pi(v_s)$,

$$\|\pi(v_s) - v_s\|_2^2 = 2 \sum_{i=1}^{n} i^2 - 2 \sum_{i=1}^{n} i\, \pi(i). \tag{54}$$

By the rearrangement inequality, the second sum is minimized by $\pi(i) = n + 1 - i$. Hence the reverse vertex is farthest from v_s, at squared distance

$$\sum_{i=1}^{n} (n + 1 - 2i)^2 = \frac{n(n^2-1)}{3}. \tag{55}$$

The Euclidean diameter of a polytope is attained by two vertices. Coordinate permutations act transitively and isometrically on the vertices of $\mathcal{P}_n$, so any such pair may be carried to v_s and another vertex. Consequently,

$$\operatorname{diam}(\mathcal{P}_n) = \|v_s - (n, n-1, \ldots, 1)\|_2 = \sqrt{\frac{n(n^2-1)}{3}}. \tag{56}$$

Substitution proves the exact formula, and its fixed-ε expansion gives the final assertion. □

4.4 Geometric Factorization of the Classical Scale

Theorem 4.4 (Dimension–Relaxation Factorization)**.** *For every fixed threshold* $\varepsilon > 0$ *and all sufficiently large n,*

$$\dim(\mathcal{P}_n) T_\varepsilon^{\max}(n) = \frac{n-1}{2} \ln \left(\frac{n(n^2-1)}{3\varepsilon^2} \right). \tag{57}$$

Consequently,

$$\dim(\mathcal{P}_n) T_\varepsilon^{\max}(n) = \frac{3}{2} n \ln n + O(n) = \Theta(n \log n). \tag{58}$$

Equivalently,

$$\dim(\mathcal{P}_n) \ln \frac{\operatorname{diam}(\mathcal{P}_n)}{\varepsilon} = \Theta(n \log n). \tag{59}$$

Proof. Proposition 3.2 gives $\dim(\mathcal{P}_n) = n - 1$, and Proposition 4.3 gives the exact relaxation time. Their product is the displayed identity. For fixed $\varepsilon > 0$,

$$\ln\left(\frac{n(n^2-1)}{3\varepsilon^2}\right) = \ln\left(\frac{n^3}{3\varepsilon^2}\left(1 - \frac{1}{n^2}\right)\right) = 3\ln n + O(1). \tag{60}$$

Multiplying by $(n-1)/2$ yields $\frac{3}{2}n\ln n + O(n)$. □

This theorem is an intrinsic geometric statement about the standard permutohedron and its quadratic relaxation, proved entirely from its dimension, Euclidean diameter, and exact contraction law. Corollary 4.5 next places this geometric quantity alongside normalized optimal comparison cost.

Let $k(n)$ denote the optimal worst-case number of comparisons required to sort n distinct elements, and define the normalized comparison clock

$$s(n) = \frac{k(n)}{n}. \tag{61}$$

Corollary 4.5 (Normalized Comparison Clock)**.** *The classical comparison bounds imply*

$$s(n) = \Theta(\log n). \tag{62}$$

Hence, for every fixed $\varepsilon > 0$,

$$s(n) = \Theta\big(T_\varepsilon^{\max}(n)\big). \tag{63}$$

Proof. The classical lower and upper bounds give $k(n) = \Theta(n\log n)$. Dividing by n gives $s(n) = \Theta(\log n)$, while Proposition 4.3 gives $T_\varepsilon^{\max}(n) = \Theta(\log n)$ for fixed $\varepsilon > 0$. □

This comparison concerns asymptotic scales. It does not assert that one unit of flow time is literally equivalent to one sweep of comparisons, and it does not replace the decision-tree proof of the comparison lower bound.

4.5 Constraint Geometry and the Ambient Flow

The exact flow in Theorem 4.2 is an ambient Euclidean benchmark. It is already compatible with the convex geometry of the permutohedron. $\mathcal{P}_n$ lies in the affine hyperplane

$$\sum_{i=1}^{n} x_i = \frac{n(n+1)}{2}, \tag{64}$$

and both $x(0)$ and v_s lie in $\mathcal{P}_n$. Since $\mathcal{P}_n$ is convex, the straight segment

$$x(t) = v_s + \big(x(0) - v_s\big)e^{-t} \tag{65}$$

remains in $\mathcal{P}_n$ for all $t \geq 0$.

It is important not to identify comparison hyperplanes with the boundary facets of $\mathcal{P}_n$. The hyperplanes

$$x_i = x_j \tag{66}$$

are braid-arrangement walls in the ambient affine space. They slice $\mathcal{P}_n$. In general, they are not facets of the permutohedron. The facets of the standard permutohedron are instead described by subset-sum inequalities [7] of the form

$$\sum_{i\in S} x_i \geq \frac{|S|(|S|+1)}{2}, \qquad \varnothing \neq S \subsetneq \{1,\dots,n\}, \tag{67}$$

together with the fixed total-sum equality. The picture thus makes a subtlety visible. Lying inside $\mathcal{P}_n$ does not by itself force any projection of the flow.

This distinction now becomes concrete. Comparison-feasible sets describe information, whereas constraints on a metric trajectory would restrict motion.

Proposition 4.6 (Projection is inactive when the target is feasible)**.** *Let $K \subseteq \mathcal{P}_n$ be a closed convex set containing v_s. For every $x \in K$,*

$$v_s - x \in T_K(x). \tag{68}$$

Here $T_K(x)$ is the tangent cone of K at x. Therefore its Euclidean projection satisfies

$$\Pi_{T_K(x)}(v_s - x) = v_s - x, \tag{69}$$

and the projected flow agrees with the ambient flow.

$$\dot{x} = v_s - x. \tag{70}$$

Proof. Since $x, v_s \in K$ and K is convex,

$$x + \lambda(v_s - x) \in K \qquad (0 \le \lambda \le 1). \tag{71}$$

Thus $v_s - x$ is a feasible tangent direction at x, so projecting it onto the tangent cone leaves it unchanged [16]. □

If a convex feasible region contains v_s, projection is therefore inactive. If instead the region excludes v_s, constrained descent approaches the unique point in the region nearest to v_s, not v_s itself. The two objects thus play distinct and complementary roles. Comparison half-spaces describe information reduction, and the Euclidean flow describes metric contraction. Comparisons remove informational ambiguity. Gradient flow removes metric distance.

4.6 Remarks

Theorem 4.2 gives exact exponential decay of Euclidean rank displacement under $V(x) = \frac{1}{2}\|x - v_s\|_2^2$. Proposition 4.3 and Theorem 4.4 extract its exact worst-case time and dimension–relaxation factorization, while Corollary 4.5 compares that geometric scale with normalized optimal comparison cost. More broadly, a descent formulation can clarify a process by identifying an energy that decreases along its path. In this respect the viewpoint echoes the insight of Jordan, Kinderlehrer, and Otto, who reinterpreted the Fokker–Planck equation as a gradient flow minimizing a free energy functional under the Wasserstein metric [17].

Figure 4 brings the discrete and continuous descriptions together. Vertices represent rank words, and the gradient flow traces a smooth curve inside the convex permutohedron toward the sorted vertex. The left panel contrasts this motion with the adjacent–swap path. Local algorithms advance by short moves along edges, while the continuous relaxation cuts straight across the polytope. This path is a genuine cross-section, not a graph-geodesic on the 1-skeleton.

In scalar form, the right panel presents the dynamics. The distance to the sorted state contracts exponentially as $r(t) = \|x(t) - v_s\|_2$ decreases. Viewed in (r, t, V)-space, the trajectory resembles motion through a curved landscape. The system rolls downhill under the potential, like a particle descending a gravitational well toward equilibrium. In both panels the example is $L = [5, 4, 2]$, used earlier. The rank word begins at $(3, 2, 1)$ and flows toward $(1, 2, 3)$, corresponding to the sorted output $[2, 4, 5]$. The initial radius is $r_0 = \|(3, 2, 1) - (1, 2, 3)\|_2 = \sqrt{8}$, and the dissipation at $t = 0$ is $dV/dt = -\|\nabla V(x(0))\|_2^2 = -8$.

Together, the panels keep the two roles visible. The left contrasts discrete and continuous paths, while the right isolates exponential metric convergence. For an initial radius above the threshold, the displayed time is $T_\varepsilon(x_0) = \ln(\|x_0 - v_s\|_2/\varepsilon)$. Its logarithmic order agrees with normalized optimal comparison cost.

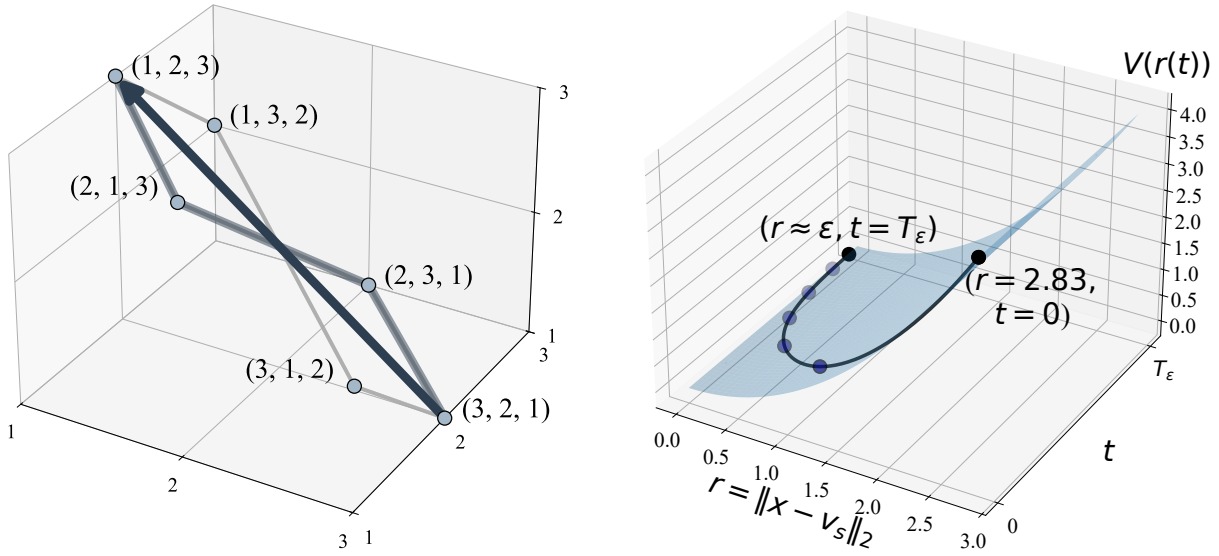

Figure 4: Gradient-flow relaxation under $V(x) = \frac{1}{2}\|x - v_s\|_2^2$. **Left panel.** Straight-line descent on $\mathcal{P}_3$ from $(3, 2, 1)$ to $(1, 2, 3)$, contrasted with the adjacent-swap path. **Right panel.** Exponential decay of $r(t) = \|x(t) - v_s\|_2$ from $r_0 = \sqrt{8}$ to a threshold ε. This Euclidean benchmark represents continuous metric relaxation, whose fixed-threshold time has the same asymptotic scale as normalized optimal comparison cost.

5 Conclusion

Efficient algorithms overcome the factorial permutation space by exploiting structure. From this perspective, the gain over local sorting appears as a contrast between local motion and information. Adjacent-swap algorithms traverse the permutohedron along its edges, resolving one inversion at a time and producing the classical $\Theta(n^2)$ behavior. Arbitrary comparisons act differently. They impose global constraints on candidate rank maps, eliminating incompatible order types and reaching the $\Theta(n \log n)$ bound. Comparison information therefore shrinks the feasible order-type set as a whole. This theme is consistent with structured decomposition methods in combinatorial algorithms [18]. It also aligns with recent enumerative work of Harris, Kretschmann, and Martínez Mori, where QuickSort comparisons are counted by parking preference lists with $n - 1$ "lucky cars" [6].

The continuous model adds the dynamical view. An ambient gradient flow on the permutohedron represents sorting as a trajectory through space and clarifies the three mechanisms at play. Adjacent swaps follow the edges of the 1-skeleton, arbitrary comparisons act through information constraints, and the Euclidean flow supplies a smooth rank-displacement benchmark inside the same polytope. It reduces squared distance to the sorted order according to

$$\|x(t) - v_s\|_2^2 = \|x(0) - v_s\|_2^2 e^{-2t}. \tag{72}$$

Whenever $0 < \varepsilon < \operatorname{diam}(\mathcal{P}_n)$, the maximal fixed-threshold relaxation time satisfies

$$T_\varepsilon^{\max}(n) = \ln \frac{\operatorname{diam}(\mathcal{P}_n)}{\varepsilon}, \tag{73}$$

and therefore, for fixed $\varepsilon > 0$,

$$\dim(\mathcal{P}_n) T_\varepsilon^{\max}(n) = \Theta(n \log n). \tag{74}$$

This is an intrinsic geometric factorization of the classical scale. The decision-tree argument remains the computational proof of the comparison lower bound, while division of optimal comparison cost by n gives the same logarithmic order as the relaxation.

Shannon's information-theoretic analysis made clear that search is governed by available information [4]. Complexity theory develops this principle across many computational settings [19]. Mann's perspective on search emphasizes the way constraints organize rather than merely restrict a space [20]. The present model gives this idea a concrete geometric form for sorting. Factorial search collapses into log-linear time when comparisons supply enough information to organize the candidate rank maps. In that sense, local traversal and global structure meet inside the permutohedron, together with its comparison half-spaces and continuous flow toward order. Informational refinement and metric contraction thus emerge as distinct but complementary geometric views of sorting. The permutohedron links them through the common asymptotic scale encoded by its dimension and Euclidean diameter.

Acknowledgment

The author used software tools, including AI assistance, for figure preparation, algebraic checks, and editing. The author is responsible for all mathematical content and conclusions.

Disclosure of interest. The author reports no conflict of interest.

Funding. No funding was received for this research.

References

[1] Donald E. Knuth. Big omicron and big omega and big theta. *SIGACT News*, 8(2):18–24, 1976.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 4th edition, 2022.

[3] Donald E. Knuth. *The art of computer programming, volume 3: sorting and searching*. Addison-Wesley, Reading, MA, 1973.

[4] Claude E. Shannon. A mathematical theory of communication. *Bell Syst Tech J*, 27:379–423, 623–656, 1948. Parts I–II; part I doi: 10.1002/j.1538-7305.1948.tb01338.x; part II doi: 10.1002/j.1538-7305.1948.tb00917.x.

[5] Guy E. Blelloch and Magdalen Dobson. The geometry of tree-based sorting. In *Proceedings of the 50th International Colloquium on Automata, Languages, and Programming (ICALP 2023)*, volume 261 of *LIPIcs*, pages 26:1–26:19, Dagstuhl, Germany, 2023. Schloss Dagstuhl–Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.ICALP.2023.26. URL https://doi.org/10.4230/LIPIcs.ICALP.2023.26.

[6] Pamela E. Harris, Jan Kretschmann, and J. Carlos Martínez Mori. Lucky cars and the quicksort algorithm. *The American Mathematical Monthly*, 131(5):417–423, 2024. doi:10.1080/00029890.2024.2309103.

[7] Günter M. Ziegler. *Lectures on polytopes*. Springer-Verlag, New York, NY, 1995.

[8] Omer Angel, Alexander E. Holroyd, Dan Romik, and Bálint Virág. Random sorting networks. *Advances in Mathematics*, 215(2):839–868, 2007. doi:10.1016/j.aim.2007.05.019.

[9] Eon Lee, Carson Mitchell, and Andrés R. Vindas-Meléndez. Stack-sorting simplices: geometry and lattice-point enumeration. *arXiv preprint*, 2023. arXiv:2308.16457.

[10] Cong Han Lim and Stephen J. Wright. Beyond the Birkhoff polytope: convex relaxations for vector permutation problems. In *Advances in Neural Information Processing Systems*, volume 27, pages 2168–2176. Curran Associates, Inc., 2014.

[11] Marco Cuturi, Olivier Teboul, and Jean-Philippe Vert. Differentiable ranking and sorting using optimal transport. In *Advances in Neural Information Processing Systems*, volume 32, pages 6861–6871. Curran Associates, Inc., 2019.

[12] Mathieu Blondel, Olivier Teboul, Quentin Berthet, and Josip Djolonga. Fast differentiable sorting and ranking. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 950–959. PMLR, 2020.

[13] Roger W. Brockett. Dynamical systems that sort lists, diagonalize matrices, and solve linear programming problems. *Linear Algebra and its Applications*, 146:79–91, 1991. doi:10.1016/0024-3795(91)90021-N.

[14] Uwe Helmke and John B. Moore. *Optimization and Dynamical Systems*. Communications and Control Engineering. Springer-Verlag, London, 1994. doi:10.1007/978-1-4471-3467-1.

[15] Persi Diaconis and R. L. Graham. Spearman's footrule as a measure of disarray. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(2):262–268, 1977. doi:10.1111/j.2517-6161.1977.tb01624.x.

[16] R. Tyrrell Rockafellar. *Convex Analysis*. Princeton University Press, Princeton, NJ, 1970.

[17] Richard Jordan, David Kinderlehrer, and Felix Otto. The variational formulation of the Fokker–Planck equation. *SIAM J Math Anal*, 29(1):1–17, 1998. doi:10.1137/S0036141096303359.

[18] John E. Hopcroft and Robert E. Tarjan. Algorithm 447: efficient algorithms for graph manipulation. *Commun ACM*, 16(6):372–378, 1973. doi:10.1145/362248.362272.

[19] Sanjeev Arora and Boaz Barak. *Computational complexity: a modern approach*. Cambridge University Press, 2009. ISBN 9780521424264.

[20] Zoltán Ádám Mann. The top eight misconceptions about NP-hardness. *Computer*, 50(5):72–79, 2017. doi:10.1109/MC.2017.146.